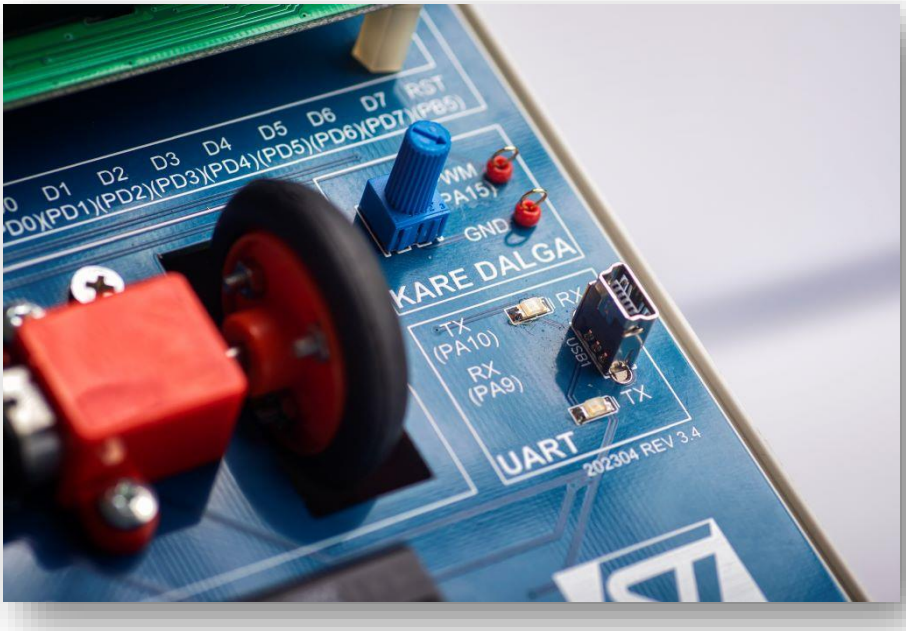
SİVAS CUMHURİYET ÜNİVERSİTESİ

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

GÖMÜLÜ SİSTEMLER DERSİ

Gömülü Sistemler Deney 6: Kablolu Haberleşme Protokolleri (UART / I2C)

PC Arayüzü ile Uzaktan Sistem Denetimi ve OLED Bilgi Paneli



Dersin Hocası: Doç. Dr. Ahmet Gürkan YÜKSEK
Dersin Asistanı: Arş. Gör. Ahmet Utku ELİK
Öğrenci Adı Soyadı – Numarası:

1. DENEYİN AMACI:

Bu deneyin temel amacı, gömülü sistem mimarilerinde yaygın olarak kullanılan kablolu haberleşme protokollerinden **UART (Universal Asynchronous Receiver-Transmitter)** ve **I2C (Inter-Integrated Circuit)** yapılarının donanımsal ve yazılımsal çalışma prensiplerini uygulamalı olarak incelemektir.

Bu kapsamda gerçekleştirilecek laboratuvar çalışması ile aşağıdaki akademik ve teknik kazanımların elde edilmesi hedeflenmektedir:

- Dış birimlerden gelen verilerin, işlemci döngüsünü (CPU Cycle) bloklamadan **UART Kesmesi (Interrupt)** mekanizması ile asenkron şekilde işlenmesi.
- **I2C protokolü** altyapısı kullanılarak SSD1306 kontrolcüsüne sahip OLED ekran modülünün sürülmesi ve sistem durum verilerinin gerçek zamanlı olarak görselleştirilmesi.
- Bilgisayar (Master/PC) ile mikrodenetleyici (Slave/STM32) arasında çift yönlü bir **komut-yanıt (Command-Response) mimarisi** kurularak uzaktan denetim sistemlerinin tasarlanması.
- Gömülü sistem ile bilgisayar entegrasyonunun sağlanması amacıyla, seri port üzerinden haberleşebilen Python tabanlı bir Grafiksel Kullanıcı Arayüzü (GUI) geliştirilmesi.

2. TEORİK ARKA PLAN:

2.1. UART (Evrensel Asenkron Alıcı-Verici)

Haberleşme protokollerinin fiziksel katman (Physical Layer) özelliklerinin anlaşılması, gömülü sistem tasarımında kritik bir öneme sahiptir. UART, asenkron bir haberleşme standardıdır; iletişim kuran dizgeler (cihazlar) arasında ortak bir saat (Clock) sinyali bulunmamaktadır. Bu nedenle veri bütünlüğünün sağlanabilmesi için alıcı ve verici birimlerin iletişim hızında (Baud Rate) önceden mutabık kalması gerekmektedir.

UART üzerinden iletilen her bir karakter, fiziksel hatta elektriksel sinyaller dizisi olarak yansıtılır ve spesifik bir "Veri Çerçevesi (Data Frame)" oluşturur.

2.2. I2C (Inter-Integrated Circuit) Protokolü

I2C, Philips (günümüzde NXP Semiconductors) tarafından geliştirilen, kart içi (on-board) ve kısa mesafeli iletişim için tasarlanmış senkron, yarı-çift yönlü (half-duplex) bir seri haberleşme protokolüdür. I2C veri yolu, mikrodenetleyiciler ile çevresel entegreler (EEPROM, RTC, Sensörler ve OLED Ekranlar) arasında sadece iki adet çift yönlü hat üzerinden iletişim kurulmasını sağlar:

- **SDA (Serial Data):** Verinin aktarıldığı hat.
- **SCL (Serial Clock):** İletişimin senkronizasyonunu sağlayan saat sinyali hattı.

Temel Özellikleri ve Çalışma Mimarisi:

- **Master-Slave (Yönetici-Köle) Mimarisi:** İletişimi her zaman "Master" cihaz (bu deneyde STM32F407) başlatır ve sonlandırır. SCL hattındaki saat sinyalini yalnızca Master cihaz üretir. "Slave" cihazlar (bu deneyde SSD1306 OLED Ekran) sadece kendilerine hitap edildiğinde cevap verirler.
- **Cihaz Adresleme:** Aynı I2C hattına paralel olarak 127 adede kadar cihaz bağlanabilir. Master cihaz veri göndermeden önce, hattın üzerine konuşmak istediği cihazın 7-bitlik (veya 10-bitlik) donanımsal donanım adresini gönderir. Sadece adresi eşleşen cihaz

iletişimi kabul eder.

- **Pull-up Dirençleri:** SCL ve SDA hatları "Açık Drenç (Open-Drain)" yapısına sahip olduğundan, hatların boştayken HIGH (Logic 1) seviyesinde kalabilmesi için VCC'ye bağlı pull-up dirençlerine ihtiyaç duyulur.

Veri Aktarım Sıralaması:

1. **START Durumu:** SCL High iken, SDA hattının Low'a çekilmesiyle iletişim başlar.
2. **Adresleme:** Master, 7-bit cihaz adresini ve hemen ardından 1-bitlik Oku/Yaz (R/W) yön bitini gönderir.
3. **ACK/NACK (Onay):** Adresi uyuşan Slave cihaz, SDA hattını Low'a çekerek "Buradayım (Acknowledge)" der.
4. **Veri Transferi:** Veriler 8 bitlik (1 Byte) paketler halinde, en anlamlı bit (MSB) ilk sırada olacak şekilde gönderilir. Her bytetan sonra alıcı cihaz bir ACK sinyali yollar.
5. **STOP Durumu:** SCL High iken, SDA hattının High'a çekilmesiyle iletişim sonlanır.

3. KULLANILACAK DONANIM VE PİN BAĞLANTILARI:

ArmApp-18 deney seti üzerinde gerçekleştirilecek "Akıllı Denetim Merkezi" uygulamasında kullanılacak çevresel birimlerin mikrodenetleyici üzerindeki pin karşılıkları ve konfigürasyon modları Tablo 1'de özetlenmiştir.

Bileşen / Hedef	**STM32 Pini**	**Kullanım Modu**	**Fonksiyonel Açıklaması**
Harici UART - TX	PA10	USART1_TX	Bilgisayara durum mesajı (String) iletmek için kullanılır. *(Modülün RX pinine bağlanacaktır)*
Harici UART - RX	PA9	USART1_RX	Bilgisayardan gelen tek baytlık komutları (Interrupt ile) alır. *(Modülün TX pinine bağlanacaktır)*
OLED Ekran - SCL	PB6	I2C1_SCL	OLED ekran iletişim senkronizasyonunu sağlar.
OLED Ekran - SDA	PB7	I2C1_SDA	OLED ekrana komut ve piksel verilerini aktarır.
Röle Modülü	PC7	GPIO_Output	Gelen 'R' komutu ile yüksek voltajlı yükleri anahtarlar.
Durum LED'i (LED0)	PE0	GPIO_Output	Gelen 'L' komutu ile görsel geri bildirim

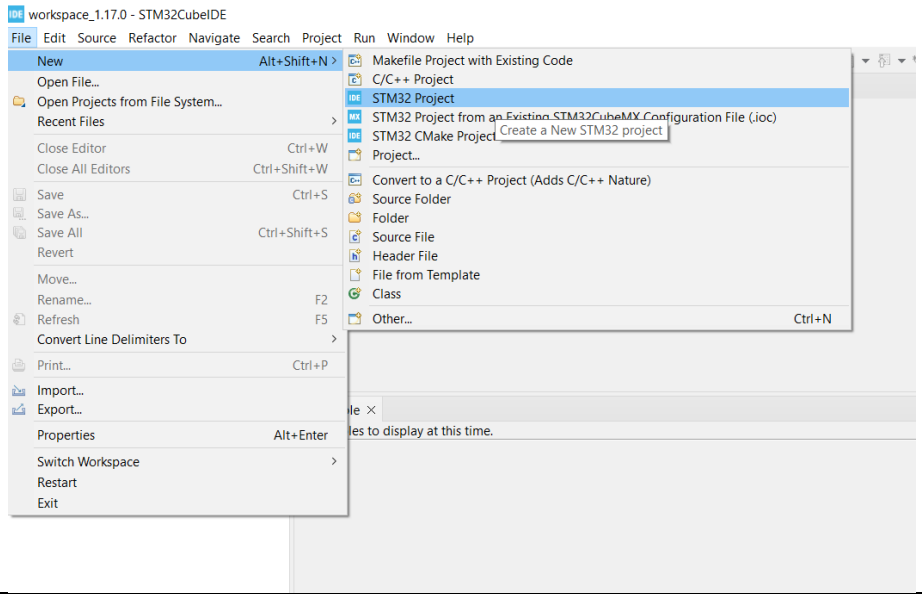
			sağlar.
Buzzer	PA8	GPIO_Output	Gelen 'B' komutu ile işitsel uyarı oluşturur.

4. STM32CubeMX KONFİGÜRASYONU (DONANIM AYARLARI)

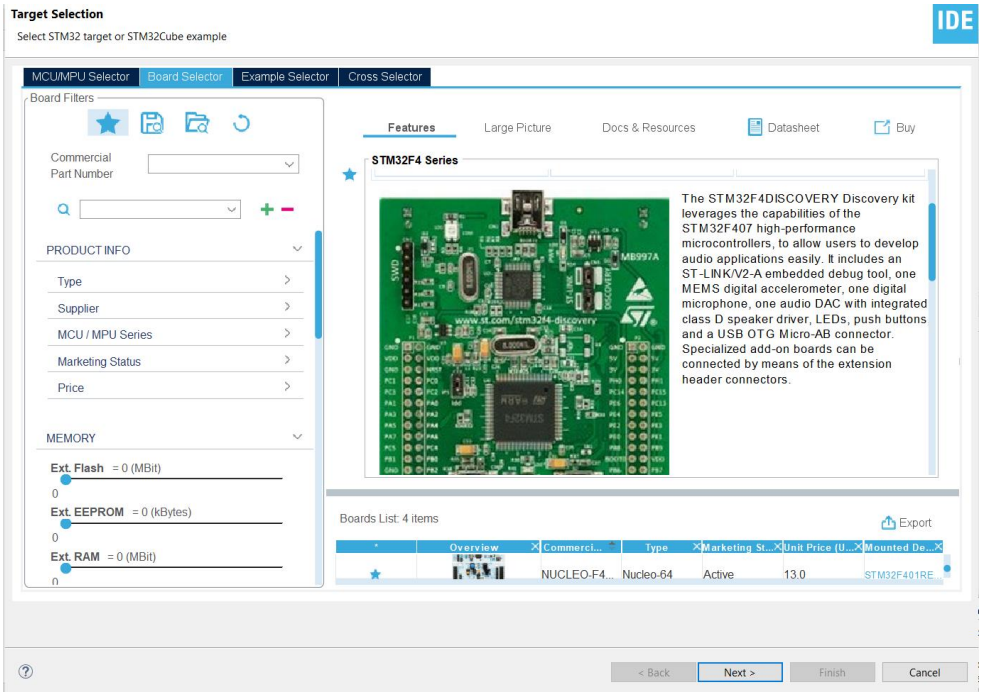
Donanım birimlerinin doğru çalışabilmesi için STM32CubeIDE içerisinde bulunan cihaz yapılandırma aracı (Device Configuration Tool) üzerinden aşağıdaki adımlar sırasıyla uygulanmalıdır.

Adım 4.1: Yeni Proje Oluşturma

1. STM32CubeIDE programını başlatın ve File > New > STM32 Project yolunu izleyin.



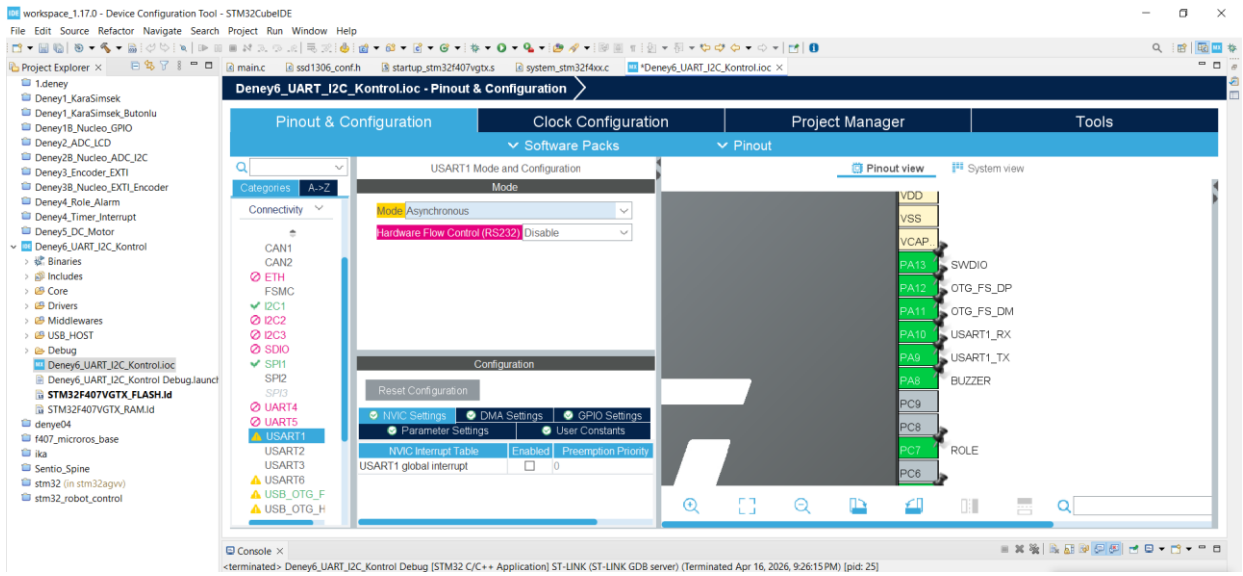
2. İşlemci seçimi ekranında (Part Number) STM32F407VGTX modelini aratın ve seçin.



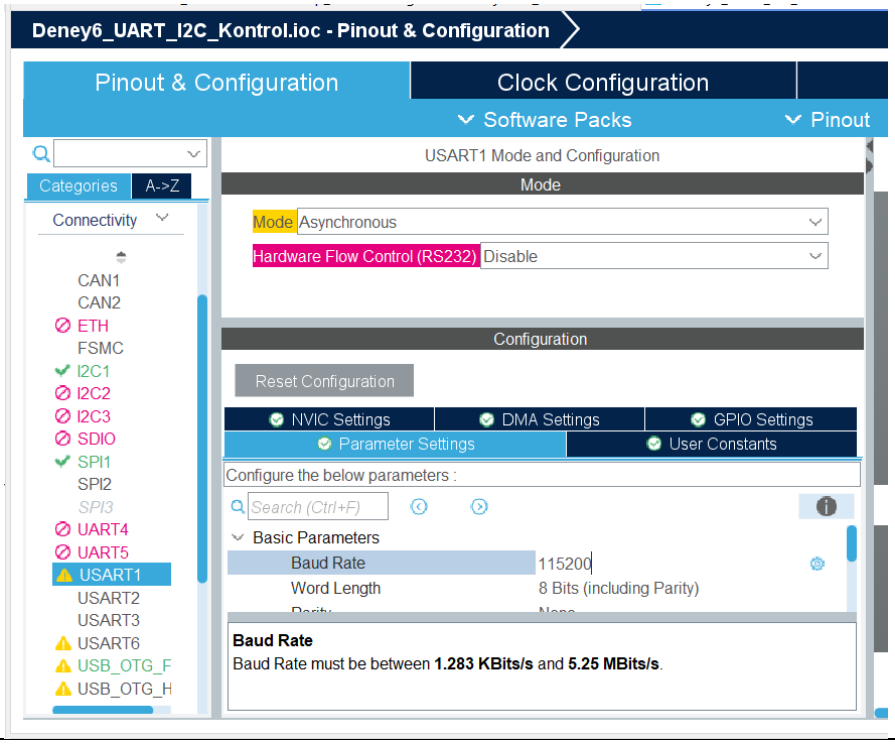
3. Proje adını Deney6_UART_I2C_Kontrol olarak isimlendirip Finish butonuna tıklayın.

Adım 4.2: USART1 (Harici UART Modülü) Yapılandırması:
Bilgisayar ile asenkron iletişim kurmak için setteki donanımsal UART modülü kullanılacaktır:

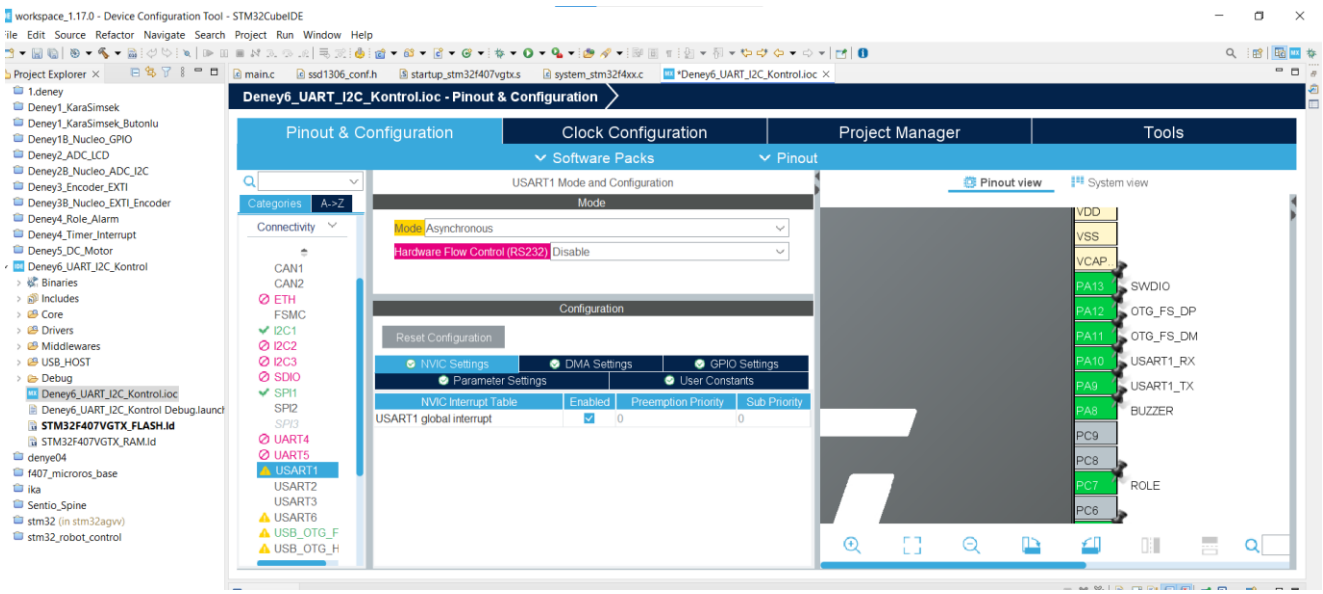
1. Sol taraftaki menüden Connectivity kategorisini genişletin. Eğer daha önceden **USART2** açıksa, tıklayıp modunu **Disable** yapın.
2. Ardından **USART1** birimine tıklayın.
3. `Mode` seçeneğini **Asynchronous** olarak ayarlayın. (İşlemci üzerinde PA9 ve PA10 pinleri yeşil renkte aktif olacaktır).



4. Alt kısımda açılan `Parameter Settings` sekmesinde `Baud Rate` değerini **115200** olarak ayarlayın.



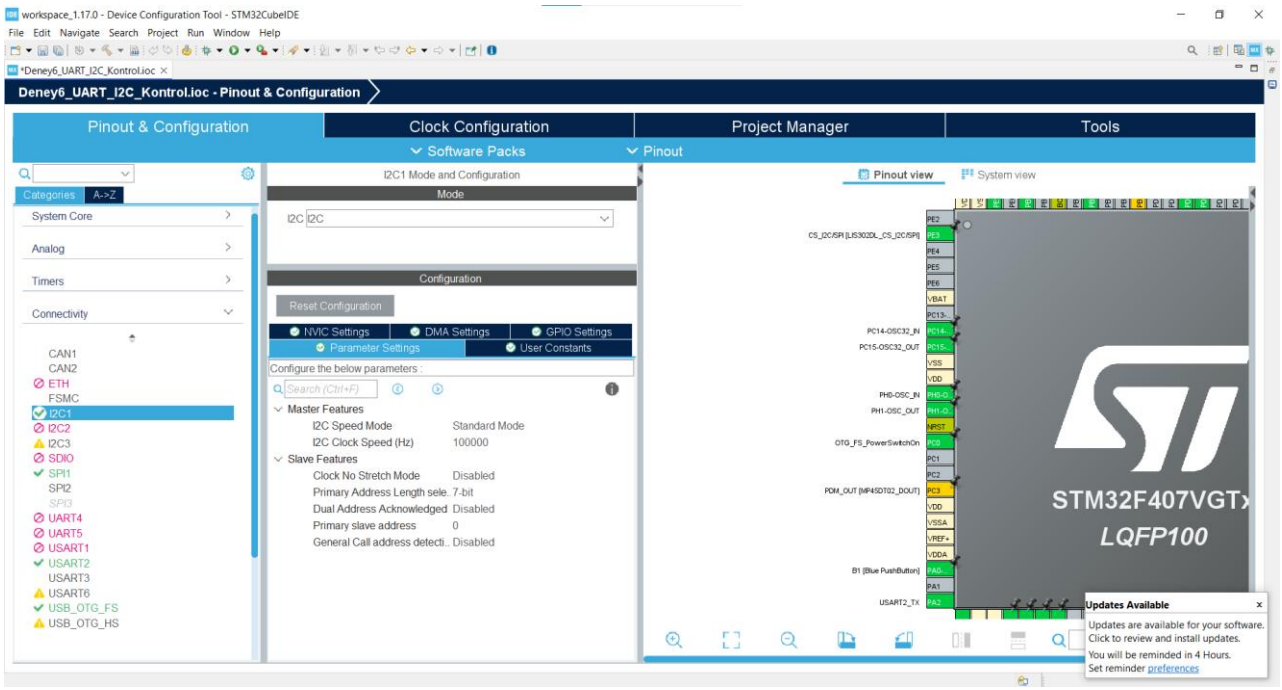
5. `NVIC Settings` sekmesine geçiş yapın ve **`USART1 global interrupt`** seçeneğinin yanındaki **`Enabled`** kutucuğunu işaretleyin. Bu işlem, dışarıdan veri geldiğinde işlemcinin uyarılmasını sağlayacaktır.



Adım 4.3: I2C1 (OLED Ekran) Yapılandırması

SSD1306 kontrolcüsüne sahip OLED ekranı sürebilmek için:

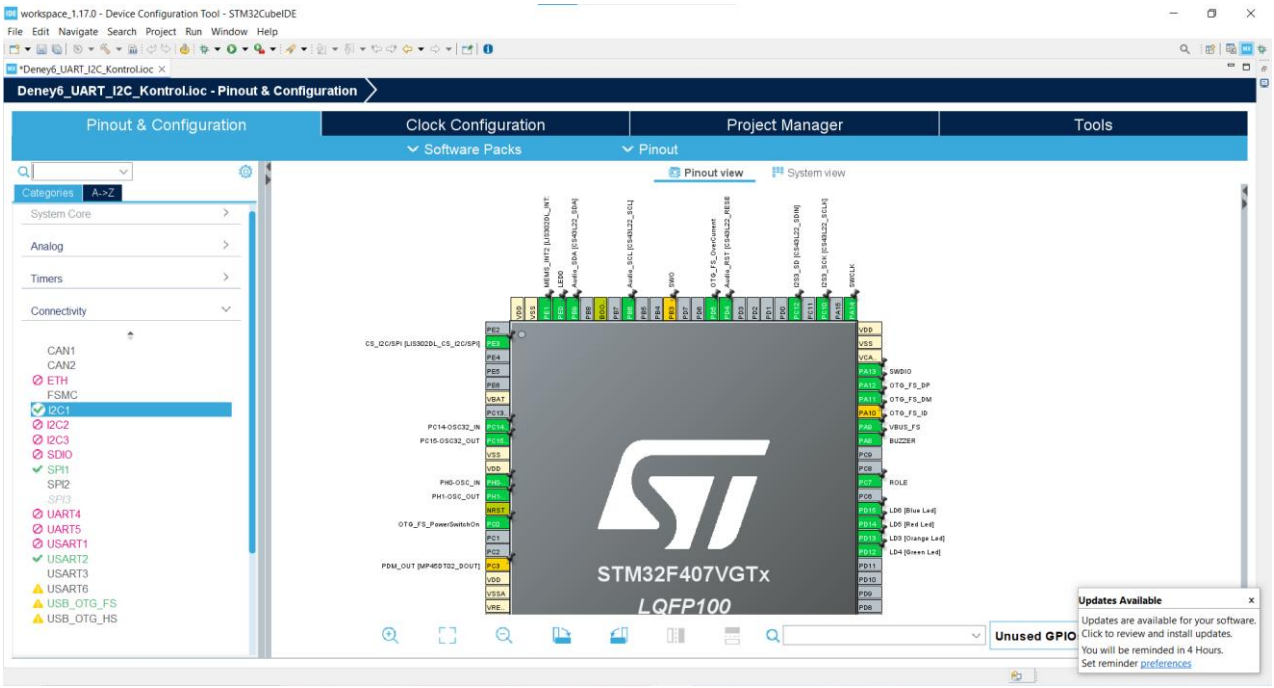
1. Connectivity kategorisi altından **I2C1** birimine tıklayın.
2. I2C modunu açılır menüden **I2C** olarak seçerek aktifleştirin. (PB6 ve PB7 pinleri otomatik olarak atanacaktır).
3. Parameter Settings sekmesinde I2C Speed Mode değerinin **Standard Mode** (100 kHz) olduğundan emin olun.



Adım 4.4: GPIO (Çıkış) Ayarları

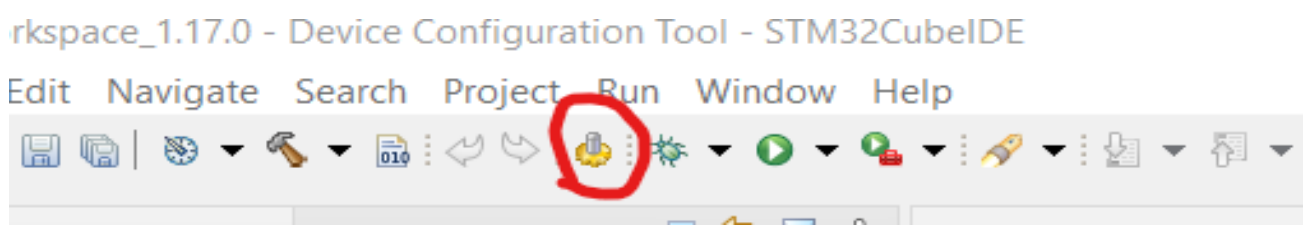
Deney setindeki fiziksel eyleyicileri (Actuators) kontrol etmek için:

1. Ekrandaki entegre grafiği (Pinout view) üzerinden sol tık ile aşağıdaki pinleri bulup **GPIO_Output** moduna alın:
   - **PC7** (Pin üzerine sağ tıklayıp Enter User Label seçeneği ile **ROLE** ismini verin).
   - **PA8** (User Label: **BUZZER**)
   - **PE0** (User Label: **LED0**)



Adım 5: Kod Üretimi (Code Generation)

- Ayarlar bittikten sonra üst menüdeki **"Device Configuration Tool Code Generation"** (Sarı dişli simgesi ⚙) butonuna basın.
- "Code is successfully generated" uyarısını aldığınızda main.c dosyanız açılacaktır.



5. YAZILIM GELİŞTİRME (MİKRODENETLEYİCİ KODLAMASI):

Donanım konfigürasyonları tamamlandıktan sonra, STM32F407 mikrodenetleyicisinin çalışma mantığını belirleyecek yazılım geliştirme aşamasına geçilir. Bu bölümde standart HAL kütüphanesi fonksiyonları ve I2C üzerinden OLED kontrolü sağlayan harici kütüphaneler (SSD1306) kullanılacaktır.

5.1. Harici OLED Kütüphanesinin Projeye Dahil Edilmesi

Karakter LCD'lerden farklı olarak, OLED ekranların her bir pikselini yönetmek karmaşık bir veri yapısı gerektirir. Bu nedenle donanım soyutlama katmanı (HAL) ile uyumlu ssd1306 kütüphanesi kullanılacaktır.

1. İlgili kütüphane dosyalarından ssd1306.h ve ssd1306_fonts.h dosyalarını projenizin **Core/Inc** klasörüne kopyalayın.
2. ssd1306.c ve ssd1306_fonts.c dosyalarını ise projenizin **Core/Src** klasörüne kopyalayın.

STM32'nin kendi kütüphanelerinde (HAL) OLED ekran için hazır bir dosya bulunmaz. Bu dosyaları açık kaynak dünyasından (GitHub) alıp projemize manuel olarak entegre etmemiz gerekiyor.
Bu kütüphane için gömülü sistemler dünyasında en çok kullanılan ve en stabil çalışan depo **"afiskon/stm32-ssd1306"** kütüphanesidir.

OLED Kütüphanesini Projeye Ekleme Adımları

1. **Kütüphaneyi İndir:**
   - Şu linke tıkla: afiskon/stm32-ssd1306 GitHub Reposu
   - Açılan sayfada sağ üstteki yeşil renkli **"Code"** butonuna tıkla ve açılan menüden **"Download ZIP"** diyerek bilgisayarına indir.

2. **Dosyaları Çıkart ve Seç:**
   - İnen ZIP dosyasını klasöre çıkart. İçinde ssd1306/ isminde bir klasör yapısı göreceksin.
   - Bize sadece şu **4 dosya** lazım:
     - ssd1306.h
     - ssd1306_fonts.h
     - ssd1306.c
     - ssd1306_fonts.c
   - *(Diğer tests veya examples klasörlerini görmezden gelebilirsin).*

3. **STM32CubeIDE'ye Aktar:**
   - **Başlık (Header) Dosyaları:** İndirdiğin klasördeki ssd1306.h ve ssd1306_fonts.h dosyalarını fareyle sürükleyip, STM32CubeIDE'deki projenin **Core/Inc** klasörünün içine bırak.
   - **Kaynak (Source) Dosyaları:** ssd1306.c ve ssd1306_fonts.c dosyalarını sürükleyip, projenin **Core/Src** klasörünün içine bırak.
   - *Not:* Sürükleyip bıraktığında IDE sana bir pencere açıp "Copy files" (Dosyaları kopyala) mi yoksa "Link to files" (Kısayol oluştur) mu diye soracaktır. Her zaman **"Copy files"** seçeneğini işaretleyip "OK" de.

4. Kütüphane Ayar Dosyasının (ssd1306_conf.h) Oluşturulması Kütüphanenin varsayılan olarak hangi işlemci ailesini (STM32F4), hangi iletişim portunu (I2C1) ve hangi yazı tiplerini (Font) kullanacağını derleyiciye bildirmek zorunludur. Aksi takdirde derleme aşamasında "undeclared" hataları alınacaktır.

1. **Core/Inc** klasörüne sağ tıklayarak **New > Header File** seçeneğini seçin.
2. Dosya adını tam olarak **ssd1306_conf.h** olarak isimlendirin.
3. Oluşturulan bu dosyanın içerisine aşağıdaki konfigürasyon kodlarını yapıştırarak kaydedin:

```
#ifndef __SSD1306_CONF_H__
#define __SSD1306_CONF_H__

// 1. İşlemci Ailesi Tanımlaması
#define STM32F4

// 2. Haberleşme Tipi ve Kullanılan Port (CubeMX'te I2C1 seçilmiştir)
#define SSD1306_USE_I2C
#define SSD1306_I2C_PORT        hi2c1

// 3. OLED Fiziksel Boyutları ve Donanım Adresi
#define SSD1306_WIDTH           128
#define SSD1306_HEIGHT          64
#define SSD1306_I2C_ADDR        0x78

// 4. Kullanılacak Yazı Tiplerinin (Font) Hafızada Aktif Edilmesi
// Bellek optimizasyonu amacıyla sadece kullanılacak fontların başındaki "//" işareti
kaldırılmalıdır.
#define SSD1306_INCLUDE_FONT_6x8
#define SSD1306_INCLUDE_FONT_7x10
#define SSD1306_INCLUDE_FONT_11x18
#define SSD1306_INCLUDE_FONT_16x26

#endif /* __SSD1306_CONF_H__ */
```

5.2. main.c Dosyası ve Kod Algoritması

Projenin ana kaynak dosyası olan main.c içerisine, belirtilen "USER CODE" bloklarına dikkat edilerek aşağıdaki kodlamalar yapılmalıdır. (Blok dışına yazılan kodlar, CubeMX güncellemelerinde silinecektir).

1. **Kütüphanelerin Tanımlanması ve Değişkenler:**

```
/* USER CODE BEGIN Includes */
#include "ssd1306.h"  // OLED Ekran sürücü kütüphanesi
#include <string.h>   // Karakter dizisi işlemleri için
#include <stdio.h>    // Standart G/Ç işlemleri için
/* USER CODE END Includes */

/* USER CODE BEGIN PV */
// UART üzerinden asenkron olarak alınacak 1 Byte'lık komutu tutan değişken
uint8_t rx_buffer;
/* USER CODE END PV */
```

2. Sistemin Başlatılması (Initialization): Ana döngü (while(1)) başlamadan önce ekranın uyandırılması ve ilk UART kesmesinin (Interrupt) kurularak işlemcinin dinleme moduna geçirilmesi gerekmektedir**.**

```
/* USER CODE BEGIN 2 */

  // --- 1. OLED Ekran Başlatma Rutini ---
  HAL_Delay(200);                        // Ekranın enerji toplaması için bekleme süresi
  ssd1306_Init();                        // I2C üzerinden ekrana başlangıç komutlarını
gönderir
  ssd1306_Fill(Black);                   // Ekran belleğini temizler (Siyah renk)

  ssd1306_SetCursor(5, 10);              // X:5, Y:10 koordinatına git
  ssd1306_WriteString("SISTEM HAZIR", Font_11x18, White);

  ssd1306_SetCursor(5, 35);
  ssd1306_WriteString("Baglanti Bekleniyor", Font_7x10, White);
  ssd1306_UpdateScreen();                // Yazılanları RAM'den fiziksel ekrana aktarır

  // --- 2. UART Kesmesinin (Interrupt) Başlatılması ---
  // USART1 üzerinden 1 byte veri geldiğinde donanım kesmesi oluşturmasını sağlar.
  HAL_UART_Receive_IT(&huart1, &rx_buffer, 1);

  /* USER CODE END 2 */
```

3. UART Kesme Servis Rutini (ISR - Interrupt Service Routine): Gömülü yazılımlarda ana döngüyü (`while(1)`) bloklamamak (Polling yapmamak) esastır. Bu nedenle veri geldiği anda tetiklenen Call-Back (Geri Çağırma) fonksiyonu kullanılır. Bu kod, dosyanın en altındaki `/* USER CODE BEGIN 4 */` bloğuna yazılmalıdır.

```
/* USER CODE BEGIN 4 */

// Bu fonksiyon, donanımsal olarak UART RX pinine veri ulaştığında otomatik olarak çalışır.
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    // Gelen kesmenin bizim beklediğimiz USART1 hattından gelip gelmediği kontrol edilir
    if (huart->Instance == USART1)
    {
        ssd1306_Fill(Black); // Yeni mesaj için ekranı temizle
        ssd1306_SetCursor(5, 5);
        ssd1306_WriteString("-- SON DURUM --", Font_11x18, White);
        ssd1306_SetCursor(5, 35);

        // Gelen komutun (rx_buffer) ASCII değerine göre analiz yapılması
        switch(rx_buffer)
        {
            case 'R': // Röle Kontrolü
                HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_7);
                ssd1306_WriteString("ROLE TETIKLENDI", Font_7x10, White);
                HAL_UART_Transmit(&huart1, (uint8_t*)"Role Konumu Degisti\n", 20, 100);
                break;

            case 'L': // LED Kontrolü
                HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_0);
                ssd1306_WriteString("LED TETIKLENDI", Font_7x10, White);
                HAL_UART_Transmit(&huart1, (uint8_t*)"LED Konumu Degisti\n", 19, 100);
                break;

            case 'B': // Buzzer Kontrolü
                HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_8);
                ssd1306_WriteString("BUZZER TETIKLENDI", Font_7x10, White);
                HAL_UART_Transmit(&huart1, (uint8_t*)"Buzzer Tetiklendi\n", 18, 100);
                break;

            default: // Bilinmeyen Komut
                ssd1306_WriteString("GECERSIZ KOMUT!", Font_7x10, White);
                HAL_UART_Transmit(&huart1, (uint8_t*)"Hatali Komut\n", 13, 100);
                break;
        }

        ssd1306_UpdateScreen(); // OLED ekranı yeni mesajla güncelle

        // İşlemciyi bir sonraki gelecek byte için tekrar dinleme moduna al (huart1)
        HAL_UART_Receive_IT(&huart1, &rx_buffer, 1);
    }
}

/* USER CODE END 4 */
```

6. BİLGİSAYAR ARAYÜZÜ GELİŞTİRME (PYTHON GUI)

Mikrodenetleyici tarafı komutları almaya hazır hale getirildikten sonra, kullanıcı deneyimini artırmak amacıyla bilgisayar tarafında bir Grafiksel Kullanıcı Arayüzü (GUI) geliştirilecektir. Terminal üzerinden ham karakter göndermek yerine butonların kullanılması, endüstriyel "SCADA" veya "HMI" sistemlerinin temelini oluşturur.

Bu işlem için Python dilinin standart kütüphanesi olan tkinter (grafik arayüz için) ve pyserial (seri haberleşme için) kullanılacaktır.

Kurulum: Komut satırından (Terminal/CMD) şu komut çalıştırılarak seri haberleşme modülü yüklenmelidir: **pip install pyserial**

Python Kaynak Kodu (`arayuz.py`):

```
import tkinter as tk
from tkinter import messagebox
import serial

# --- 1. SERİ PORT (UART) KONFİGÜRASYONU ---
# DİKKAT: 'COM5' değeri bilgisayarınızın Aygıt Yöneticisindeki (Device Manager)
# STLink Virtual COM Port numarasına göre değiştirilmelidir.
PORT_NUMARASI = 'COM5'
BAUD_RATE = 115200

try:
    # Seri portu başlat ve 1 saniyelik zaman aşımı (timeout) belirle
    ser = serial.Serial(PORT_NUMARASI, BAUD_RATE, timeout=1)
except Exception as e:
    print(f"KRİTİK HATA: Seri porta bağlanılamadı. Hata Kodu:\n{e}")
    exit()

# --- 2. HABERLEŞME FONKSİYONU ---
def komut_ileti(cmd):
    """
    Kullanıcının tıkladığı butona ait karakteri (R, L veya B) STM32'ye gönderir.
    Ardından STM32'den gelen doğrulama mesajını (ACK) okuyup arayüze yazdırır.
    """
    try:
        # Karakteri byte dizisine çevir (encode) ve gönder
        ser.write(cmd.encode())

        # STM32'den gelen '\n' karakterine kadarki mesajı oku ve string'e çevir (decode)
        cihaz_yaniti = ser.readline().decode('utf-8').strip()

        # Arayüzdeki bilgi etiketini güncelle
        lbl_bilgi.config(text=f"Cihaz Yanıtı: {cihaz_yaniti}", fg="#2ecc71")
    except Exception as e:
        lbl_bilgi.config(text=f"İletişim Koptu!", fg="#e74c3c")

# --- 3. GRAFİKSEL KULLANICI ARAYÜZÜ (GUI) TASARIMI ---
pencere = tk.Tk()
pencere.title("ArmApp-18 HMI Paneli")
pencere.geometry("380x350")
pencere.configure(bg="#2c3e50") # Arka plan rengi (Koyu Lacivert)

# Ana Başlık
lbl_baslik = tk.Label(pencere, text="SİSTEM DENETİM MERKEZİ", font=("Verdana", 14, "bold"),
bg="#2c3e50", fg="white")
lbl_baslik.pack(pady=20)

# Butonlar (Her buton tıklandığında 'komut_ileti' fonksiyonunu farklı parametreyle çağırır)
btn_role = tk.Button(pencere, text="⚡ Röle Modülü (Toggle)", font=("Verdana", 11, "bold"),
bg="#e74c3c", fg="white", width=25, height=2, command=lambda: komut_ileti('R'))
btn_role.pack(pady=5)
```

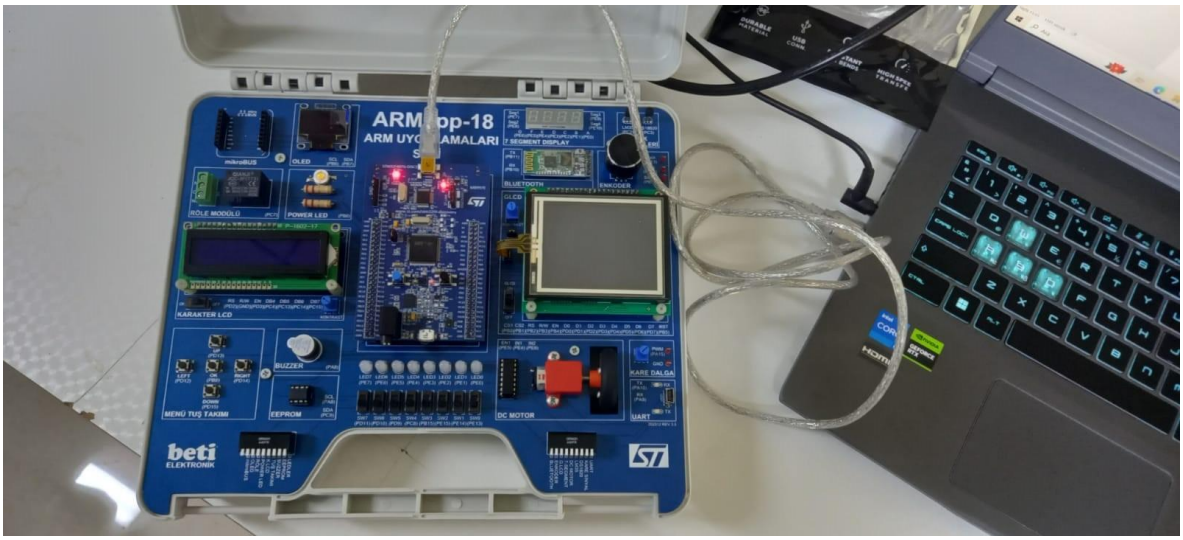
```
btn_led = tk.Button(pencere, text="💡 Durum LED (Toggle)", font=("Verdana", 11, "bold"),
bg="#f1c40f", fg="black", width=25, height=2, command=lambda: komut_ileti('L'))
btn_led.pack(pady=5)

btn_buzzer = tk.Button(pencere, text="🔊 Sesli Uyarı (Buzzer)", font=("Verdana", 11, "bold"),
bg="#3498db", fg="white", width=25, height=2, command=lambda: komut_ileti('B'))
btn_buzzer.pack(pady=5)

# STM32 Geri Bildirim Göstergesi
lbl_bilgi = tk.Label(pencere, text="Cihaz Yanıtı: Sistem Hazır", font=("Verdana", 10,
"italic"), bg="#2c3e50", fg="#bdc3c7")
lbl_bilgi.pack(pady=25)

# Arayüz döngüsünü başlat
pencere.mainloop()
```

7. Pratik Uygulama ve Test Adımları

Yazdığımız kodun STM32 kartına yüklenmesi ve deney seti üzerinde test edilmesi için aşağıdaki adımları izleyin.

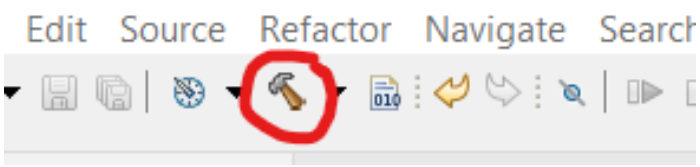
Adım 1: Fiziksel Bağlantı

1. ArmApp18 deney seti üzerindeki **STM32F4 Discovery** kartının üst kısmında bulunan **mini-USB (USB ST-LINK)** girişine kabloyu takın.
2. Kablonun diğer ucunu bilgisayarınızın USB portuna bağlayın.
3. Kart üzerindeki **PWR** (Güç) LED'inin yandığını ve bilgisayarınızın donanımı tanıdığını doğrulayın.

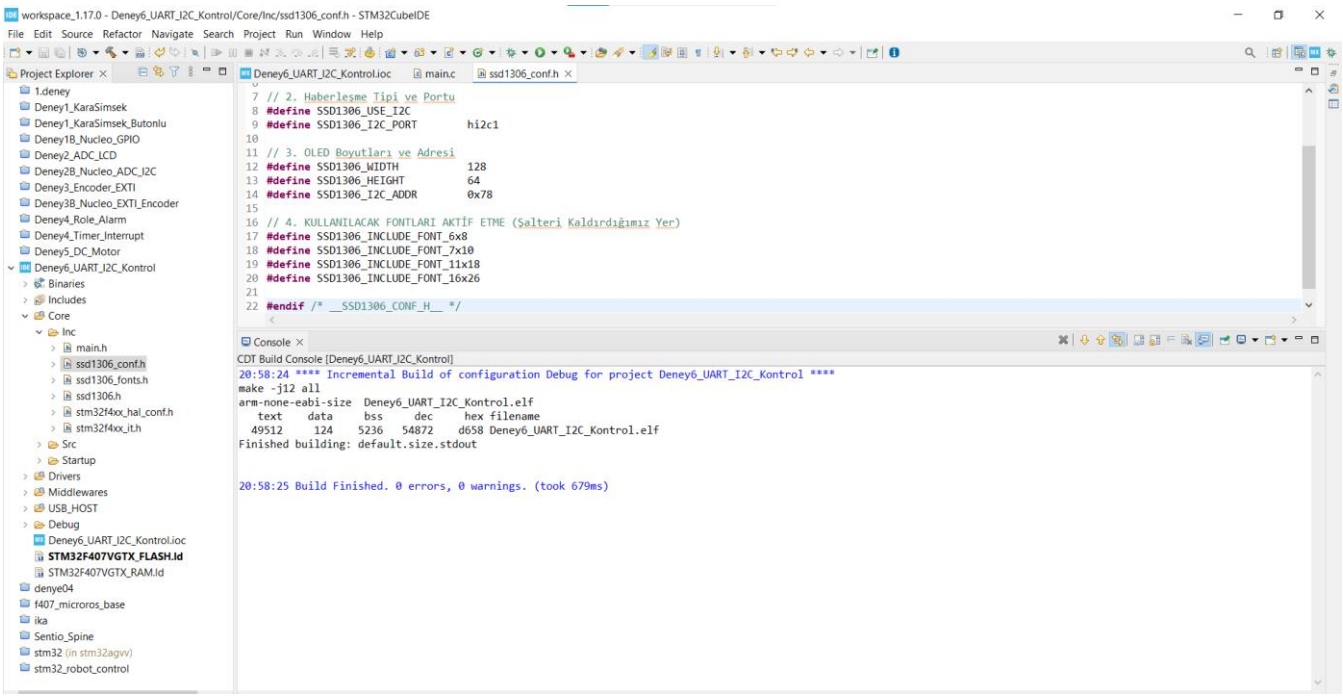


Adım 2: Derleme ve Yükleme (Build & Run)

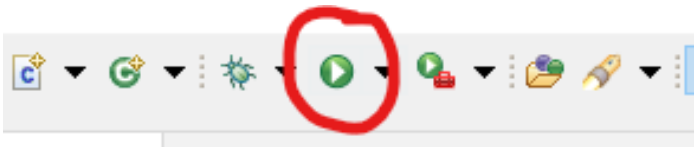
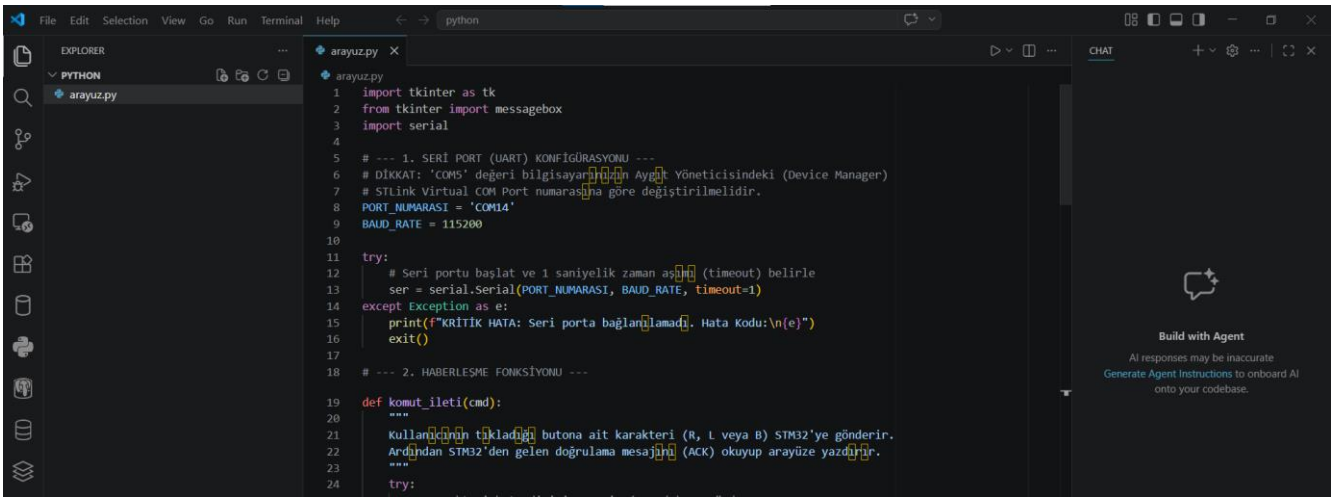
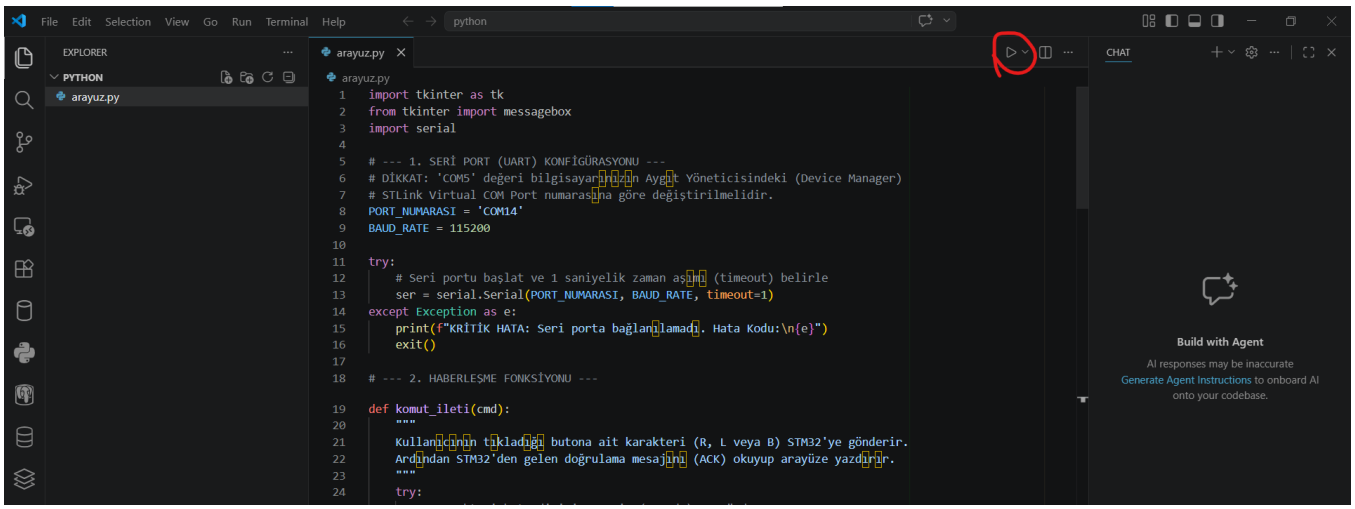
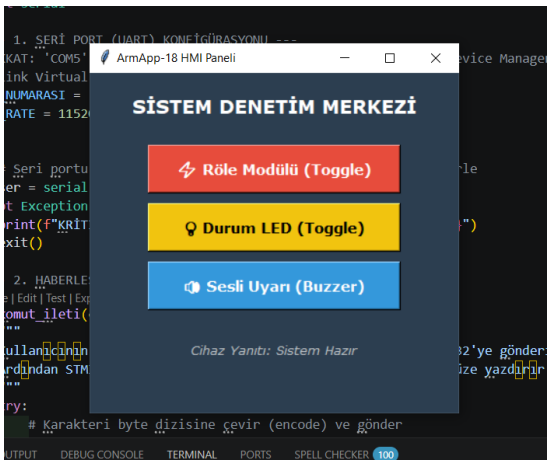
1. **Derleme:** STM32CubeIDE üst menüsündeki **"Çekiç" (Build)** ikonuna tıklayın (veya Ctrl+B).



- *Console penceresinde "0 errors, 0 warnings" mesajını görmelisiniz.*



2. **Yükleme:** Derleme hatasız ise, yeşil **"Oynat" (Run)** ikonuna tıklayın.



Arayüz için vs code da :